

Reprendre la main sur mon backend NodeJS

<https://github.com/jsorant/kata-refacto-backend-node>



Jérémie Sorant

Application backend

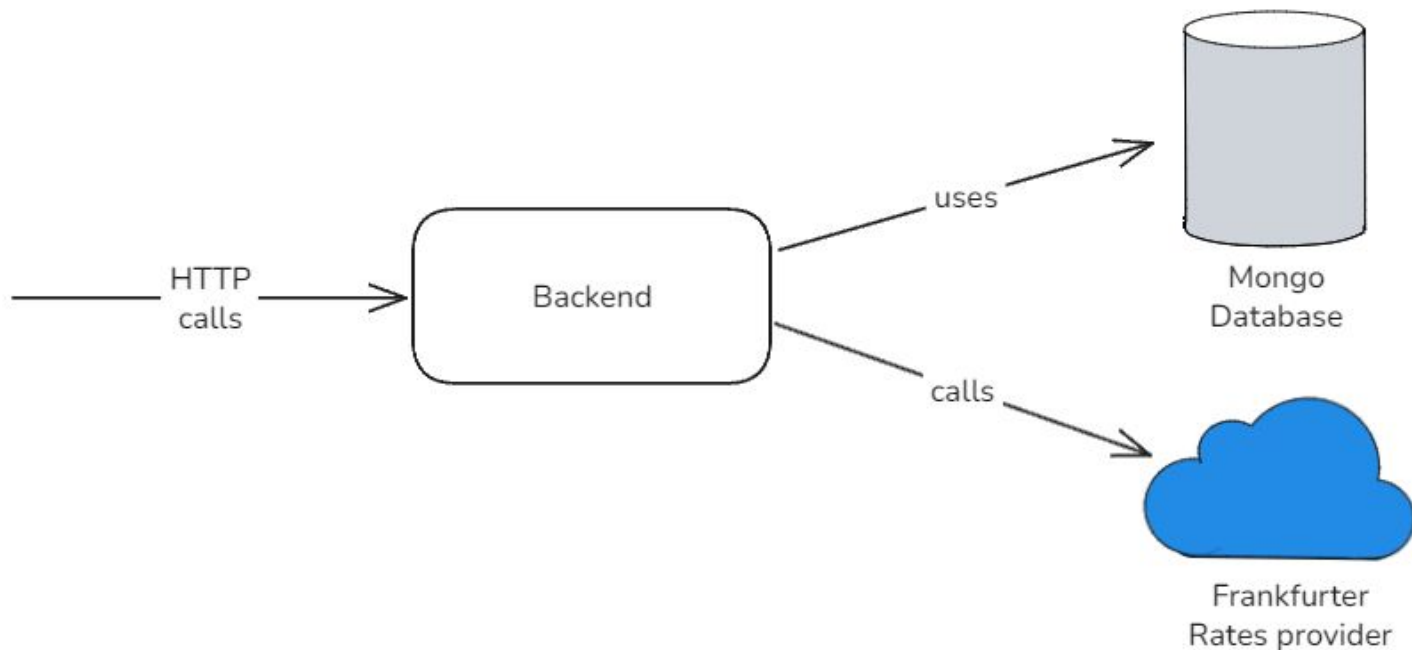
Gère des comptes bancaires

Features :

- Créer un compte
- Déposer de l'argent
- Retirer de l'argent
- Consulter le solde (deux devises : EUR et JPY)



Ça ressemble à ça :



Les problèmes



Code pas clair



Tests manuals & feedback long



Augmentation de la dette



Objectifs

Reprendre la main sur le code

→ Utiliser différentes typologies de test

- Filet de sécurité

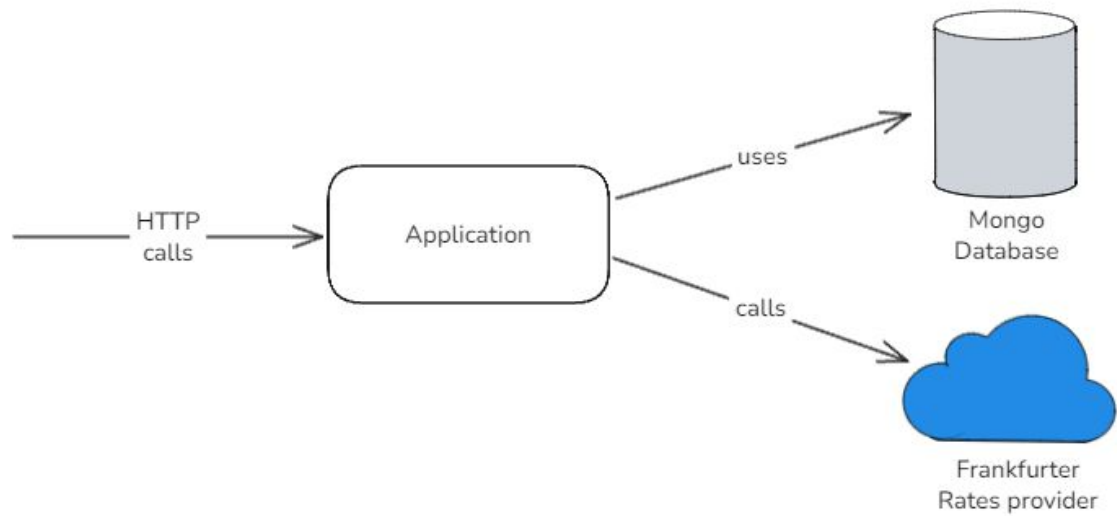
- Feedback adapté / DX

→ Ajouter une feature (TDD double loop)

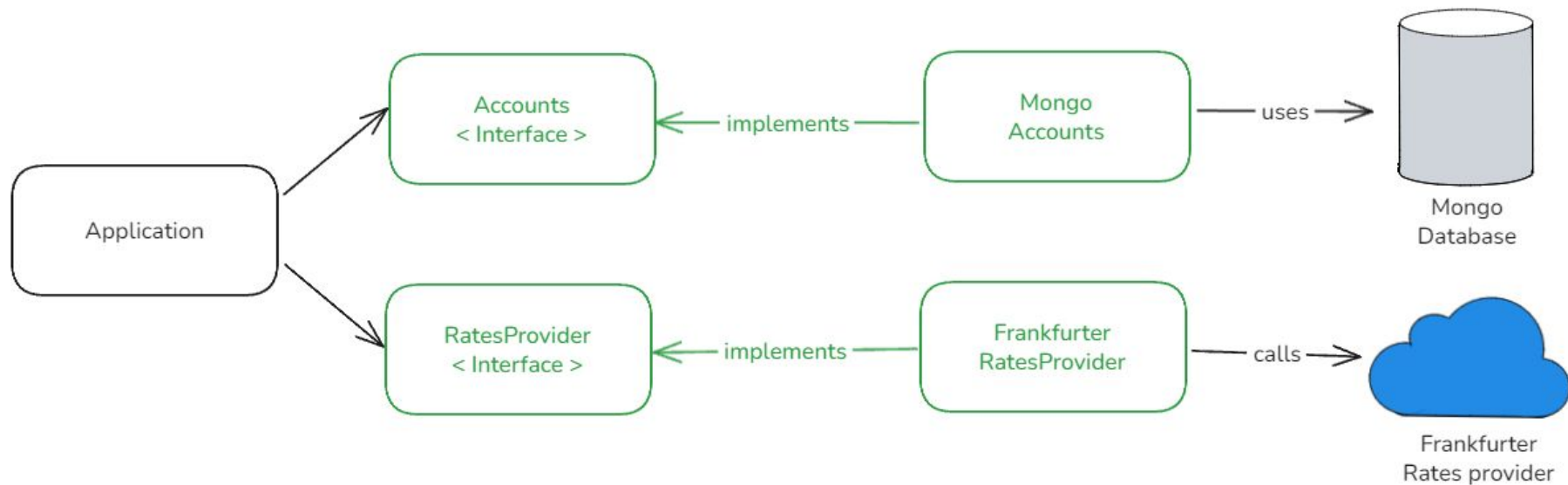
Live coding !

Bilan

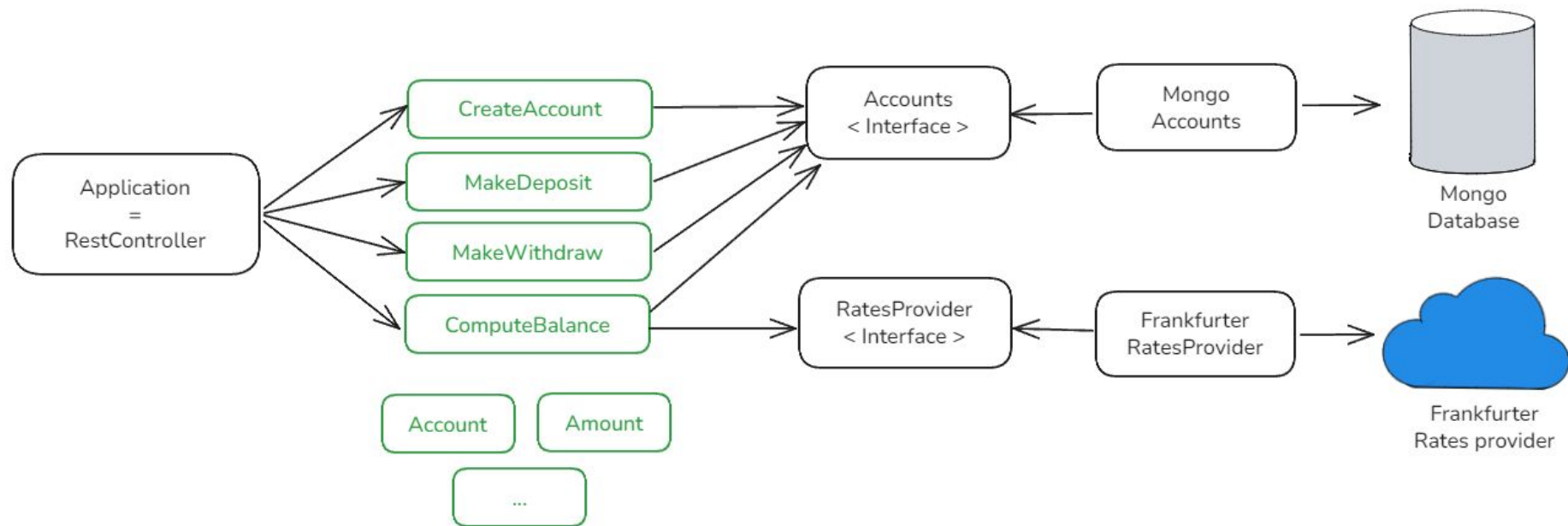




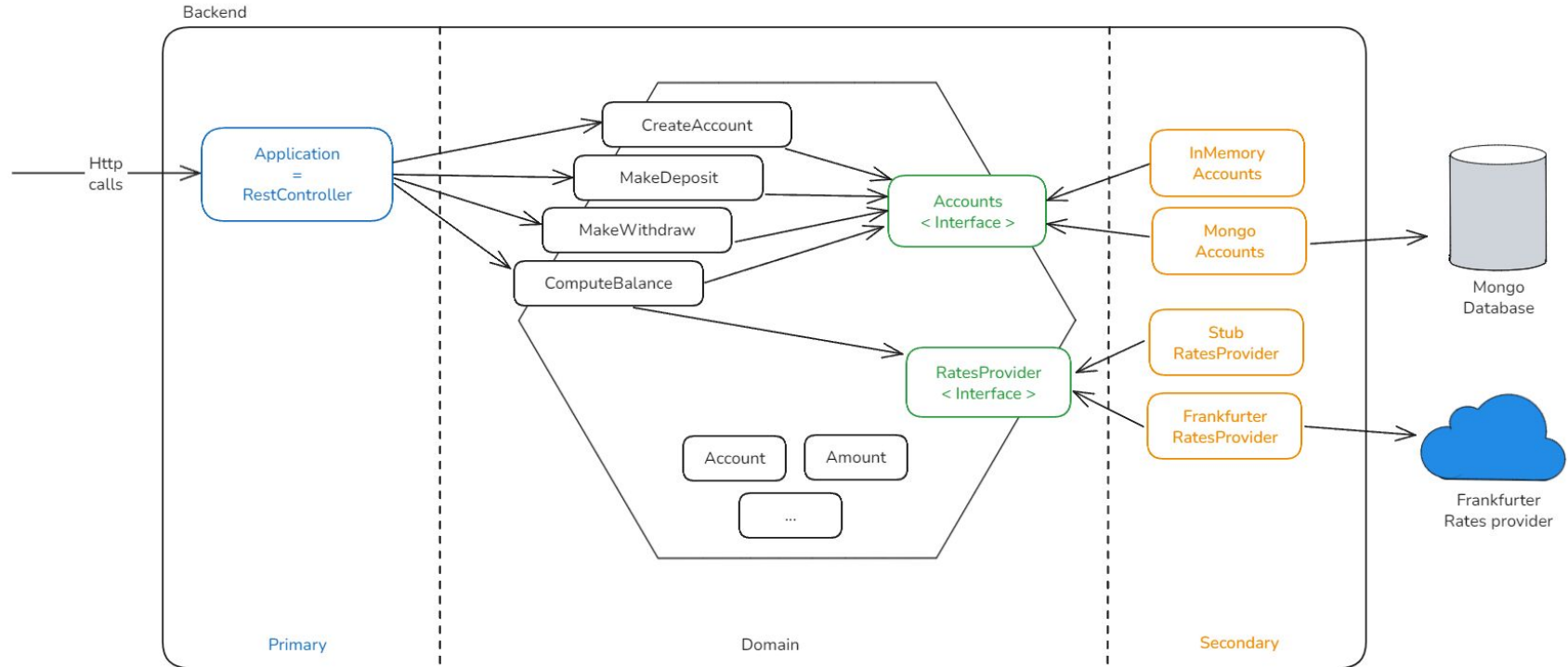
Isoler les dépendances externes



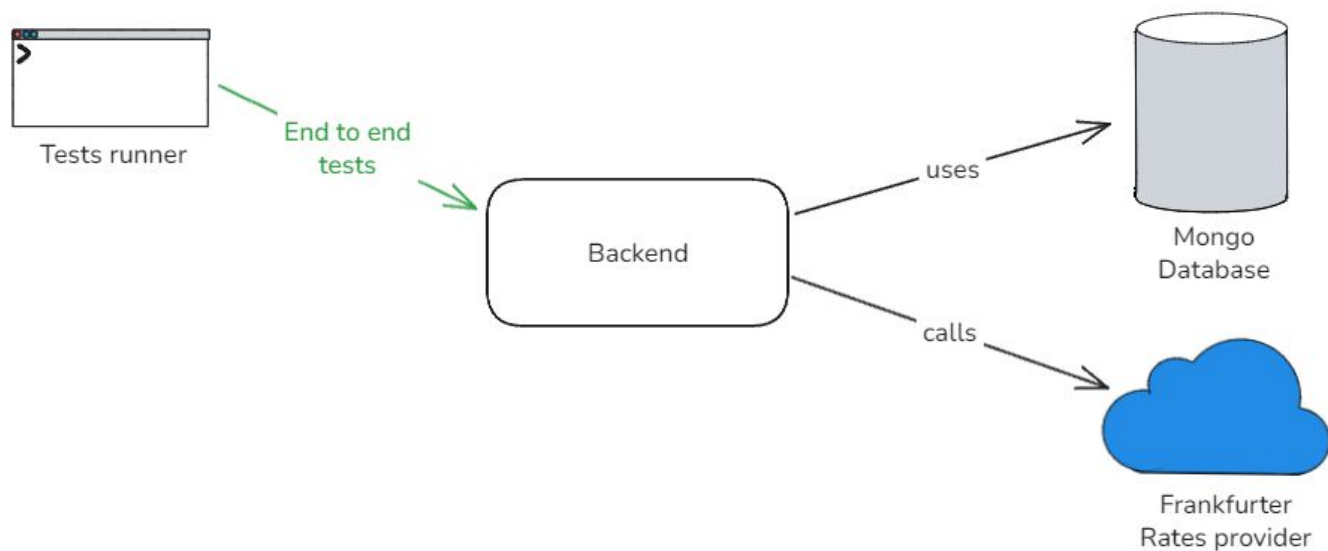
Isoler les règles métier



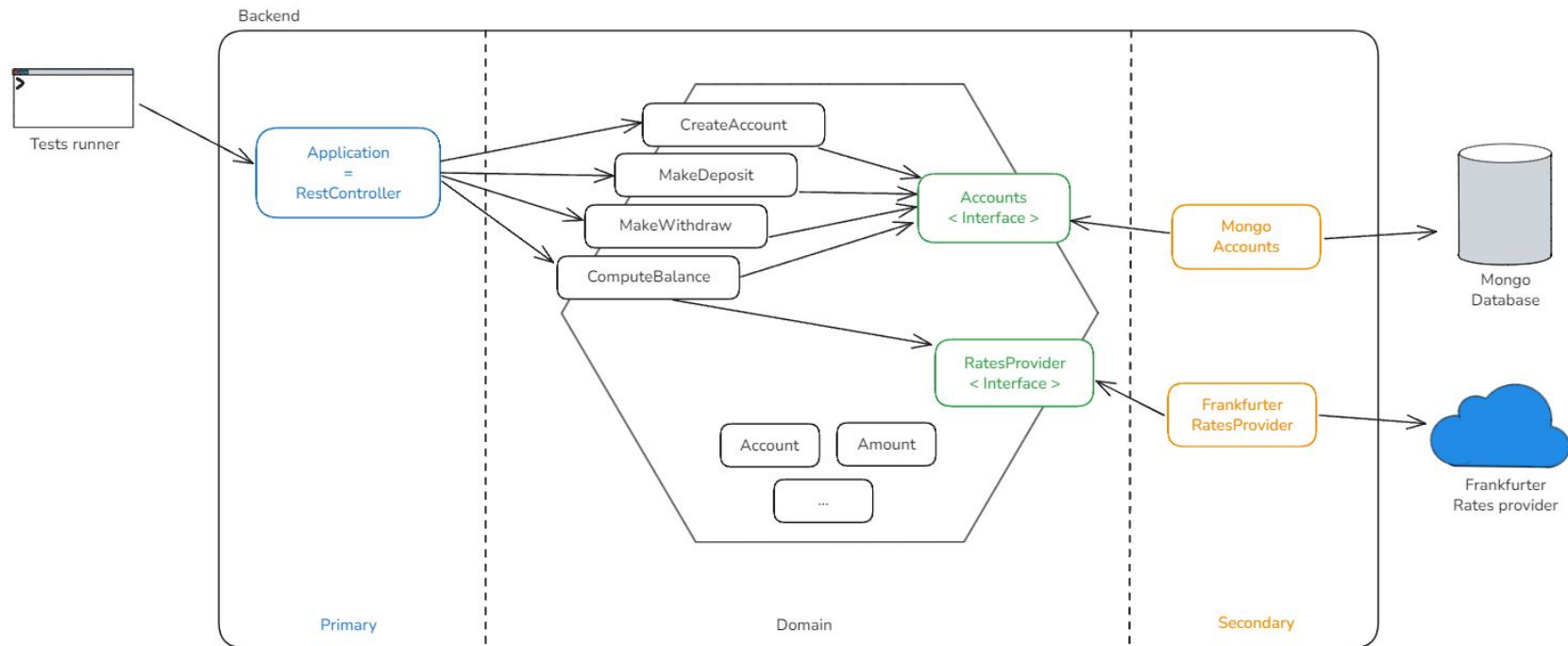
Architecture hexagonale



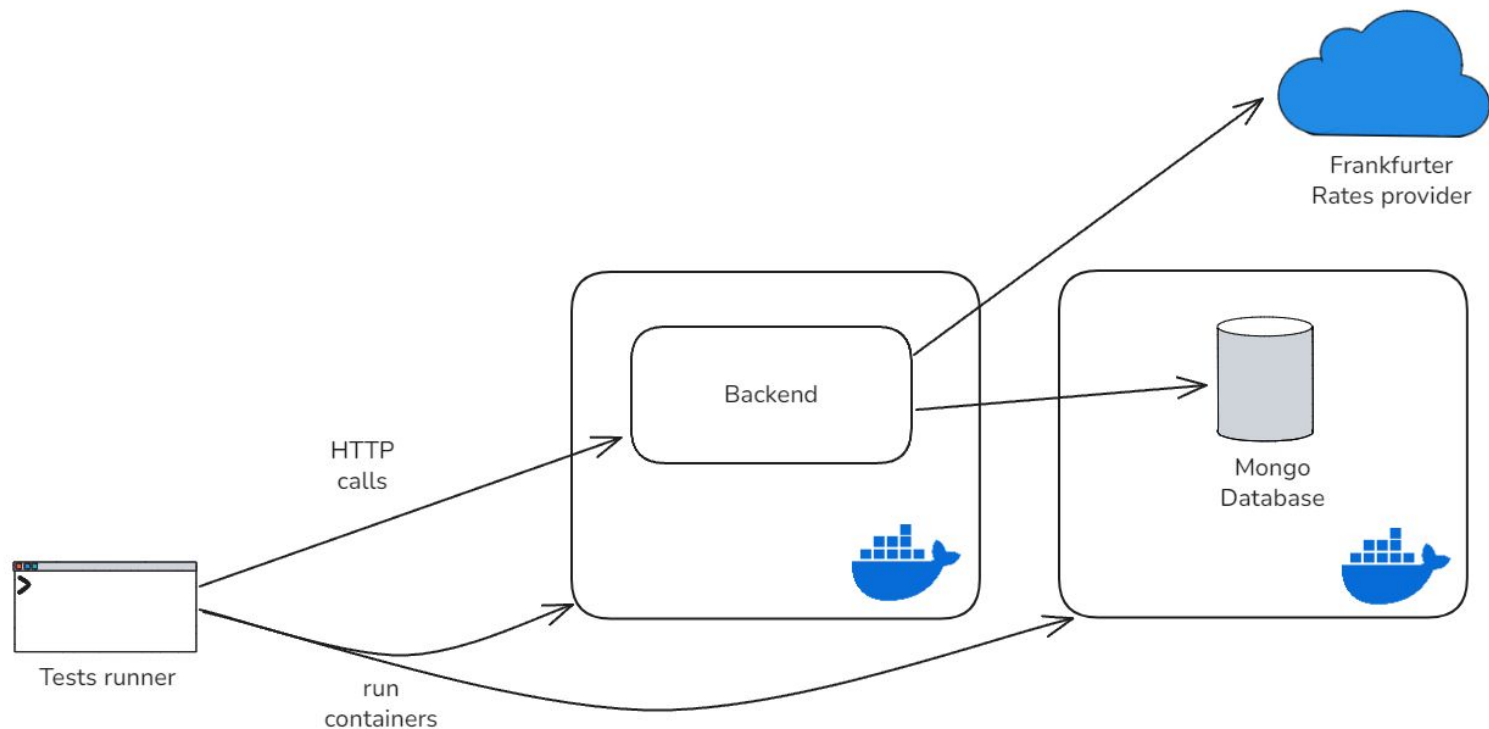
Tests end to end



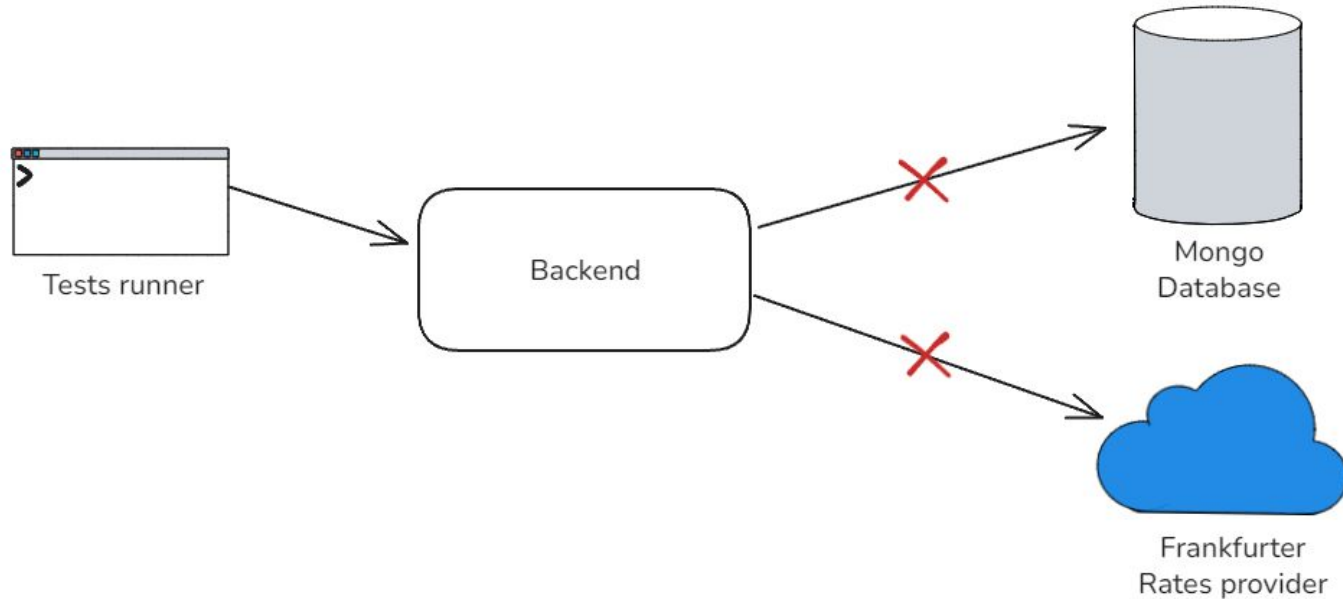
Tests end to end



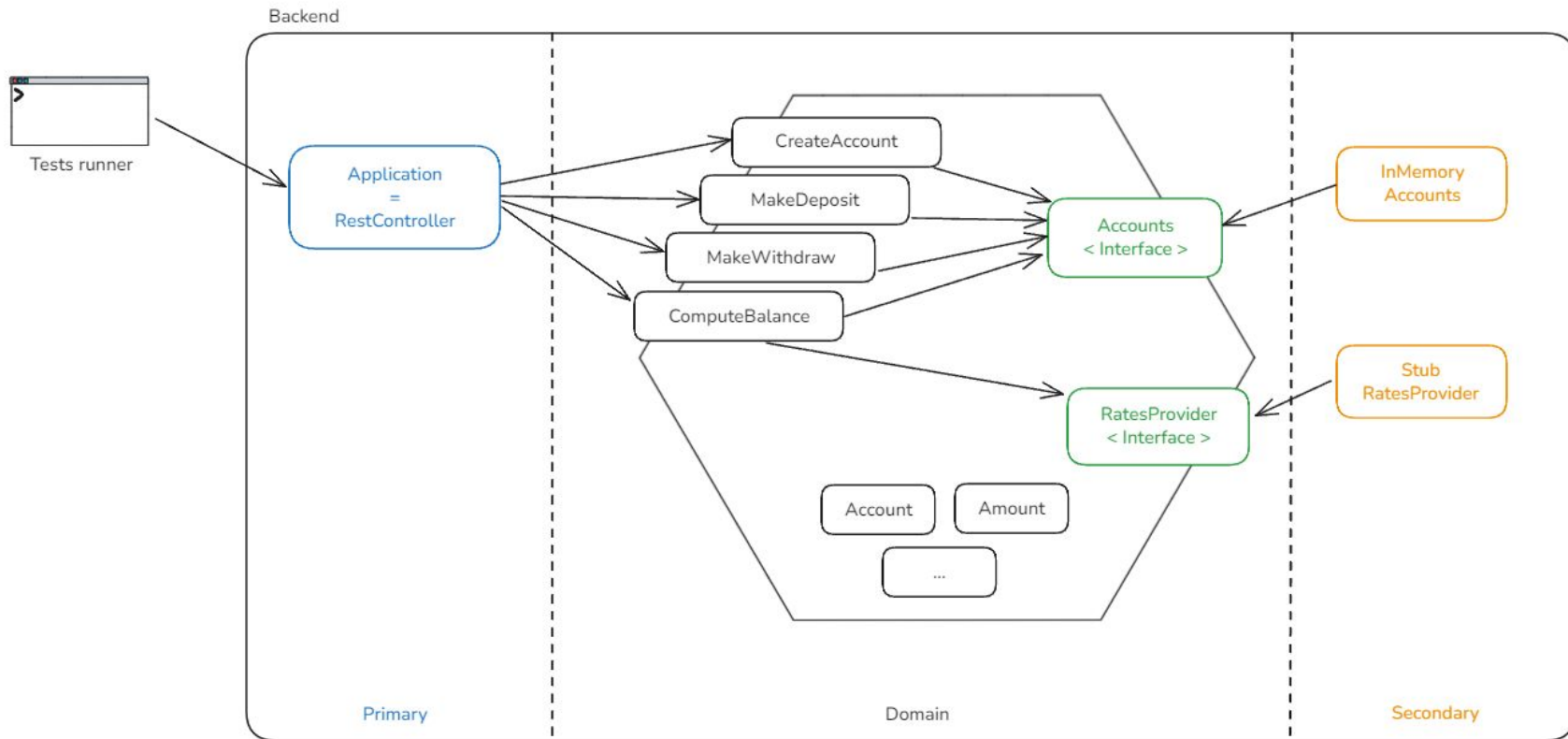
Tests end to end



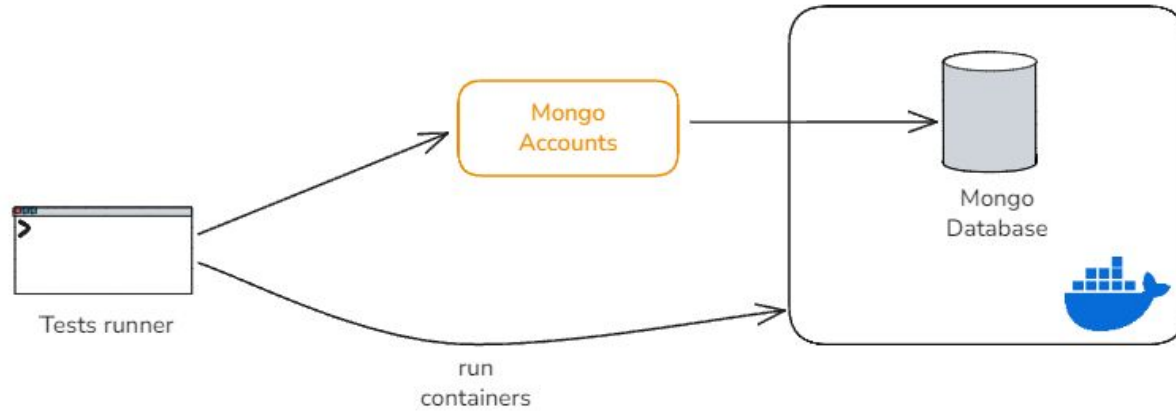
Tests de composant



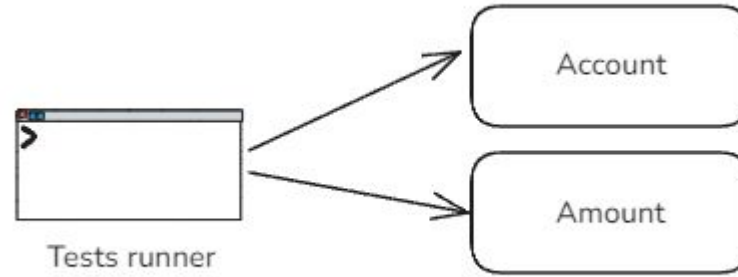
Tests de composant



Tests d'intégration



Tests “unitaires” - grain fin



Outils

- Tests runner : Vitest, mode UI, code coverage, watcher mode
- Supertest
- TestContainers



Concepts

- Doublures de test (stub / fake)
- SRP (SOLID)
- DIP (SOLID)
- Injection de dépendance
- Screaming architecture
- Double loop TDD



Ouvertures

- Stratégies de tests : pyramid, diamond, honeycomb...
- Mieux gérer les dates et le générateur de UUID
- Primitive obsession
- Immutability
- Aggregate / Entity / Value Object
- Event sourcing / CQRS
- ...





CRAFTING
SUSTAINABLE
CODE WE DO



Jérémie Sorant

Développeur chez SHODO Lyon

Co-organisateur des Software Crafters Lyon



[linkedin.com/in/jeremy-sorant](https://www.linkedin.com/in/jeremy-sorant)



github.com/jsorant

COACHING . CRAFT . JUSTICE SOCIALE

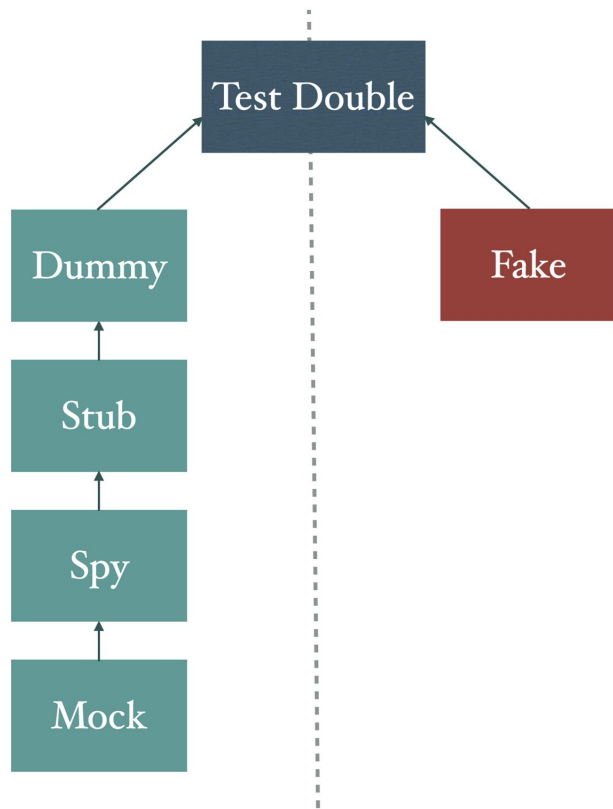
 shodo.io



 SHODOioFR

SHODO

Doublures de test



Dates

```
export class Deposit extends Transaction {  
  constructor(date: Date, amount: Amount) {  
    super(date, amount);  
  }  
  
  static of(amount: Amount): Deposit {  
    return new Deposit(new Date(), amount);  
  }  
  
  applyTo(balance: number): number {  
    return balance + this.amount.value;  
  }  
}
```

```
export class Withdraw extends Transaction {  
  constructor(date: Date, amount: Amount) {  
    super(date, amount);  
  }  
  
  static of(amount: Amount): Withdraw {  
    return new Withdraw(new Date(), amount);  
  }  
  
  applyTo(balance: number): number {  
    return balance - this.amount.value;  
  }  
}
```

GUID

```
private static generateAccountId() : string {  
    return `account-${randomUUID()}`;  
}
```

Primitive obsession

```
export class Amount {  
  readonly value: number;  
  
  constructor(value: number) {  
    this.ensureValueIsPositive(value);  
    this.value = value;  
  }  
  
  private ensureValueIsPositive(value: number) : void {  
    if (value < 0) throw new Error("Amount must be positive!");  
  }  
}
```

Immutability

```
addDeposit(amount: Amount): void {  
    this.transactions.push(Deposit.of(amount));  
}  
  
addWithdraw(amount: Amount): void {  
    this.ensureCanWithdraw(amount);  
    this.transactions.push(Withdraw.of(amount));  
}
```

Comportement vs implémentation

Le comportement est stable

L'implémentation est instable



29



LYON
GRA 47-33

J'écris des tests sans pleurer maintenant

1 k vues • il y a 5 mois



Software Crafters Lyon

On nous dit que les tests automatisés, c'est la clé de la qualité